Q1. Given an array containing all elements as duplicate except 1 element. Find that element.

ex ⇒ arr ⇒ { 1 1 2 6 3 6 2 4 4 }

XOR of all elements

XOR → { 1 1 2 2 4 4 6 6 3 }

0 0 0 0 3

3

O/p = 3.

$$\boxed{\begin{array}{l} TC \Rightarrow O(N) \\ SC \Rightarrow O(1) \end{array}}$$

Q2. Given N elements, every element repeats twice, except 2 unique, find 2 unique element

freq = 1

ex ⇒ arr[6] = { 3, 6, 4, 4, 3, 8 } ⇒ O/p ⇒ 6, 8

arr[4] ⇒ { 4 9 9 8 } ⇒ O/p = 4, 8.

Idea 1

get freq of each element using hashmap and return the two elements having freq = 1.

TC => O(N)

SC => O(N)

Idea 2

Sort the array, iterate and find nos. which are not duplicates.

TC => O(N log N)

SC => O(1) → depends on sort algo.

Idea 3

arr => { 3, 6, 4, 4, 3, 8 }

XOR of all elements

XOR of all elements = 6 ^ 8 = 14

∴ [XOR of all elements = XOR of 2 unique nos] arr[] => 10 (1010), 8 (1000), 8 (1000), 9 (1001), 12 (1100), 9 (1001), 6 (0110), 11 (1011), 10 (1010), 6 (0110), 12 (1100), 17 (10001)

XOR of all elements = 11^17

	4	3	2	1	0
11 :	0	1	0	1	1
17 :	1	0	0	0	1
11^17 =>	1	1	0	1	0

obs:
at bit pos: 1, 3, 4
both unique elements (11 & 17) have different bits.

Bit pos (1) → reference

set → Element having a set bit, at bit pos. 1
{10, 6, 11, 10, 6,}
→ XOR of all
11

unset → Element having an unset bit, at bit pos. 1
{8, 8, 9, 12, 9, 12, 17}
→ XOR of all
17

arr[] => 10 (1010), 8 (1000), 8 (1000), 9 (1001), 12 (1100), 9 (1001), 6 (0110), 11 (1011), 10 (1010), 6 (0110), 12 (1100), 17 (10001)

Bit pos (3) → reference

set
{10 8 8 9
12 9 11 10
12}
→ XOR of all
11

unset
{6 6 17}
→ XOR of all
17

pseudo

i) XOR of all elements

```
val = 0
for( i=0; i<N; i++){
    val = val ^ arr[i]
}
```

⇒ O(N)

ii) find first set bit in val

```
pos = -1
for( i=0; i<32; i++){
    if( checkBit(val, i)){
        pos = i
        break;
    }
}
```

O(32) ←

↓

O(1)

iii) find two unique elements

```
set = 0, unset = 0
for( i=0; i<N; i++){
    if( checkBit( arr[i], pos )){
        set ^= set ^ arr[i]
    else {
        unset = unset ^ arr[i]
}
```

O(N) ←

iv) return ans ⇒ { unset, set }

v) keyboard drop

$$\boxed{\begin{array}{l} TC \Rightarrow O(N) \\ SC \Rightarrow O(1) \end{array}}$$

Q3. Given N array elements, array contains all elements from [1 to N+2] except 2 element. Find the missing 2 elements.

arr ⇒ {3, 6, 1, 4} ⇒ O/P = 2, 5

$N = 4$

1 to 6

arr ⇒ {1, 6, 4, 7, 5} ⇒ O/P ⇒ 2, 3.

$N = 5$

1 to 7

Idea 1 : sort and find missing elements

TC ⇒ $O(N \log N)$

SC ⇒ $O(1)$ → depends on sort algo.

Idea 2 : Hashmap / boolean array [N+2]

TC ⇒ $O(N)$

SC ⇒ $O(N)$

Idea 3

1 to 7 ⇒ {1, 2, 3, 4, 5, 6, 7}

arr[5] ⇒ {1, 6, 4, 7, 5}

← approach Q1

TC ⇒ O(N)
SC ⇒ O(1)

→ pseudo → H(W

Q4: Given N elements, calculate sum of XOR of all pairs.

arr[5] ⇒ {3, 5, 6, 8, 2}

brute force

```
sum=0
for(i=0; i<N; i++){
    for(j=0; j<N; j++){
        sum = sum + (arr[i] ^ arr[j])
    }
}
```

TC ⇒ O(N2)
SC ⇒ O(1)

idea

try to reduce redundant operations

arr[5] => { 3, 5, 6, 8, 2 }

3^3	3^5	3^6	3^8	3^2
5^3	5^5	5^6	5^8	5^2
6^3	6^5	6^6	6^8	6^2
8^3	8^5	8^6	8^8	8^2
2^3	2^5	2^6	2^8	2^2

$\boxed{\begin{array}{l} TC \Rightarrow O(N^2) \\ SC \Rightarrow O(1) \end{array}}$

Optimisation

iterate on upper part & get sum

result = 2 * sum

	2^3 (3)	2^2 (2)	2^1 (1)	2^0 (0)					
3^5 →	0	1	1	0	→	0	2^2	2^1	0
3^6 →	0	1	0	1	—	0	2^2	0	2^0
3^8 →	1	0	1	1	—	2^3	0	2^1	2^0
3^2 →	0	0	0	1	—	0	0	0	2^0
5^6 →	0	0	1	1	—	0	0	2^1	2^0
5^8 →	1	1	0	1	—	2^3	2^2	0	2^0
5^2 →	0	1	1	1	—	0	2^2	2^1	2^0
6^8 →	1	1	1	0	—	2^3	2^2	2^1	0
6^2 →	0	1	0	0	—	0	2^2	0	0
8^2 →	1	0	1	0	—	2^3	0	2^1	0
	4	6	6	6		4	6	6	6

final $\Rightarrow \left[4 \times 2^3 + 6 \times 2^2 + 6 \times 2^1 + 6 \times 2^0\right] \times 2$

Obs for any i^{th} bit pos, if C_i no. of pairs for which i^{th} bit is set

contribution $\Rightarrow \left[C_i \times 2^i\right]$

$(i = 0 \to N-1)$

$(j = 0 \to N-1)$

pair $\Rightarrow$ i

$C_i \times 2^i$

TC $\Rightarrow$ $O(31 N^2)$

3, 5, 6, 8, 2

2: 0010	3: 0011	5: 0101	6: 0110	8: 1000

(bit positions 3 2 1 0 written above each)

	set	unset	
bit pos $\to$ 0	2	3	$\Rightarrow$ 6 pairs $\times 2^0$
bit pos $\to$ 1	3	2	$\Rightarrow$ 6 pairs $\times 2^1$
bit pos $\to$ 2	2	3	$\Rightarrow$ 6 pairs $\times 2^2$

bit pos → 3   1   4   ⇒   4 pairs $\times 2^3$

pseudo

```
sum = 0
for(i=0; i<=31; i++){
    c = 0 // no. of elements with set bit at pos i
    for(j=0; j<N; j++){
        if( checkBit( arr[j], i) ){
            c++
        }
    }
    sum = sum + [ (c) * (N-c) * (1<<i) ]     // 2^i
}
return 2 * sum;
```

$$\boxed{\begin{aligned} TC &= O(31 * N) = O(N) \\ SC &= O(1) \end{aligned}}$$

Q Given N check if its power of 2.

H.W

- no loops
- no inbuilt func
- no math. func.

$N = 2$ ✓

$N = 16$ ✓

$N = 20$ ✗

$\boxed{TC = O(1)}$

$2 \rightarrow$	10
$16 \rightarrow$	10000
$8 \rightarrow$	1000
$32 \rightarrow$	100000

only 1 bit as set

$20 \rightarrow$	10100
$18 \rightarrow$	10010

multiple set bits